FIAP / Alura
Pós-Graduação em IA para Devs
Tech Challenge Fase 3

ASSISTENTE VIRTUAL MÉDICO BASEADO EM LLM COM FINE-TUNING, RAG E ORQUESTRAÇÃO DE FLUXOS INTELIGENTES

Alunos:
Adriel Felipe Cândido Santos
João Marcos Simões
Letícia Rodrigues Nepomucena Lopes
Lucas Felipe Silva
Vagner Barbosa Dantas

SUMÁRIO

1. Introdução
2. Fundamentação Teórica
3. Metodologia
4. Arquitetura do Sistema
5. Desenvolvimento
6. Fluxo de Funcionamento
7. Avaliação e Resultados
8. Discussão
9. Conclusão
10. Trabalhos Futuros
11. Referências

---

1. INTRODUÇÃO

O avanço dos modelos de linguagem de grande escala (LLMs) tem possibilitado a criação de sistemas inteligentes capazes de compreender e gerar linguagem natural com alto nível de sofisticação. No domínio da saúde, tais modelos apresentam potencial significativo para apoio à triagem de sintomas, orientação inicial e disseminação de informações médicas.

Entretanto, modelos generalistas apresentam limitações relevantes quando aplicados a contextos médicos, como baixa especialização terminológica e risco de geração de informações imprecisas.

Este trabalho tem como objetivo desenvolver um assistente virtual médico generalista, combinando técnicas de fine-tuning, recuperação de informação (RAG) e orquestração de fluxos inteligentes, visando maior precisão, rastreabilidade e segurança.

---

2. FUNDAMENTAÇÃO TEÓRICA

2.1 Modelos de Linguagem (LLMs)

LLMs são modelos treinados em grandes volumes de texto, capazes de realizar tarefas como geração de texto, resposta a perguntas e sumarização.

2.2 Fine-Tuning

O fine-tuning consiste na adaptação de um modelo pré-treinado para um domínio específico, utilizando dados especializados. No contexto médico, essa técnica melhora significativamente a qualidade das respostas.

2.3 LoRA e QLoRA

LoRA (Low-Rank Adaptation) é uma técnica eficiente que reduz o custo computacional do treinamento ao modificar apenas partes do modelo.

QLoRA complementa essa abordagem com quantização em 4 bits, reduzindo ainda mais o uso de memória sem perda significativa de desempenho.

2.4 Retrieval-Augmented Generation (RAG)

RAG combina modelos generativos com mecanismos de recuperação de informação, permitindo que o modelo utilize conhecimento externo atualizado e confiável.

---

3. METODOLOGIA

Este projeto caracteriza-se como uma pesquisa aplicada, com abordagem experimental e foco na implementação de um sistema funcional.

Etapas principais:

1. **Coleta de dados médicos via scraping:** Extração de protocolos clínicos do CONITEC/MS, artigos do TelessaúdeRS e templates do RadReport.
2. **Preparação e estruturação dos dados:** Unificação em formato JSONL, padronização de campos (instruction, input, output), validação de qualidade, anonimização e divisão em datasets de treino (80%) e validação (20%).
3. **Fine-tuning do modelo:** Adaptação dos modelos BioMistral-7B e TinyLlama com a técnica QLoRA.
4. **Implementação do sistema RAG:** Desenvolvimento do mecanismo de busca com TF-IDF e, opcionalmente, embeddings.
5. **Orquestração do fluxo conversacional:** Utilização do LangGraph para gerenciar o estado da conversa e rotear as requisições do usuário.
6. **Avaliação dos resultados:** Análise de métricas de perplexidade, similaridade de respostas e relevância do RAG.

---

4. ARQUITETURA DO SISTEMA

A arquitetura do sistema foi projetada de forma modular, composta por diferentes camadas responsáveis por garantir escalabilidade, organização e separação de responsabilidades.

Os principais componentes incluem:

- **Workflow conversacional baseado em grafos (LangGraph):** Gerencia o fluxo da conversa, classificando a intenção do usuário e direcionando para a ferramenta apropriada.
- **Pipeline de processamento de linguagem natural (LangChain):** Orquestra a interação entre o usuário, o sistema RAG e o LLM.

- **Sistema de recuperação de informação (RAG):** Indexa e busca em uma base de conhecimento de documentos médicos para fornecer contexto relevante.
- **Base de dados de pacientes (SQLite):** Armazena prontuários simulados para contextualizar as respostas com o histórico do paciente.
- **Camada de validação e segurança:** Realiza a sanitização das entradas para prevenir ataques (XSS, Injection) e identifica potenciais emergências.

O sistema utiliza roteamento condicional para tratar diferentes tipos de entrada, como perguntas médicas, situações de emergência e interações gerais, permitindo respostas mais adequadas ao contexto.

A Figura 1 apresenta uma visão geral da arquitetura implementada, evidenciando os principais módulos e seus fluxos de interação.

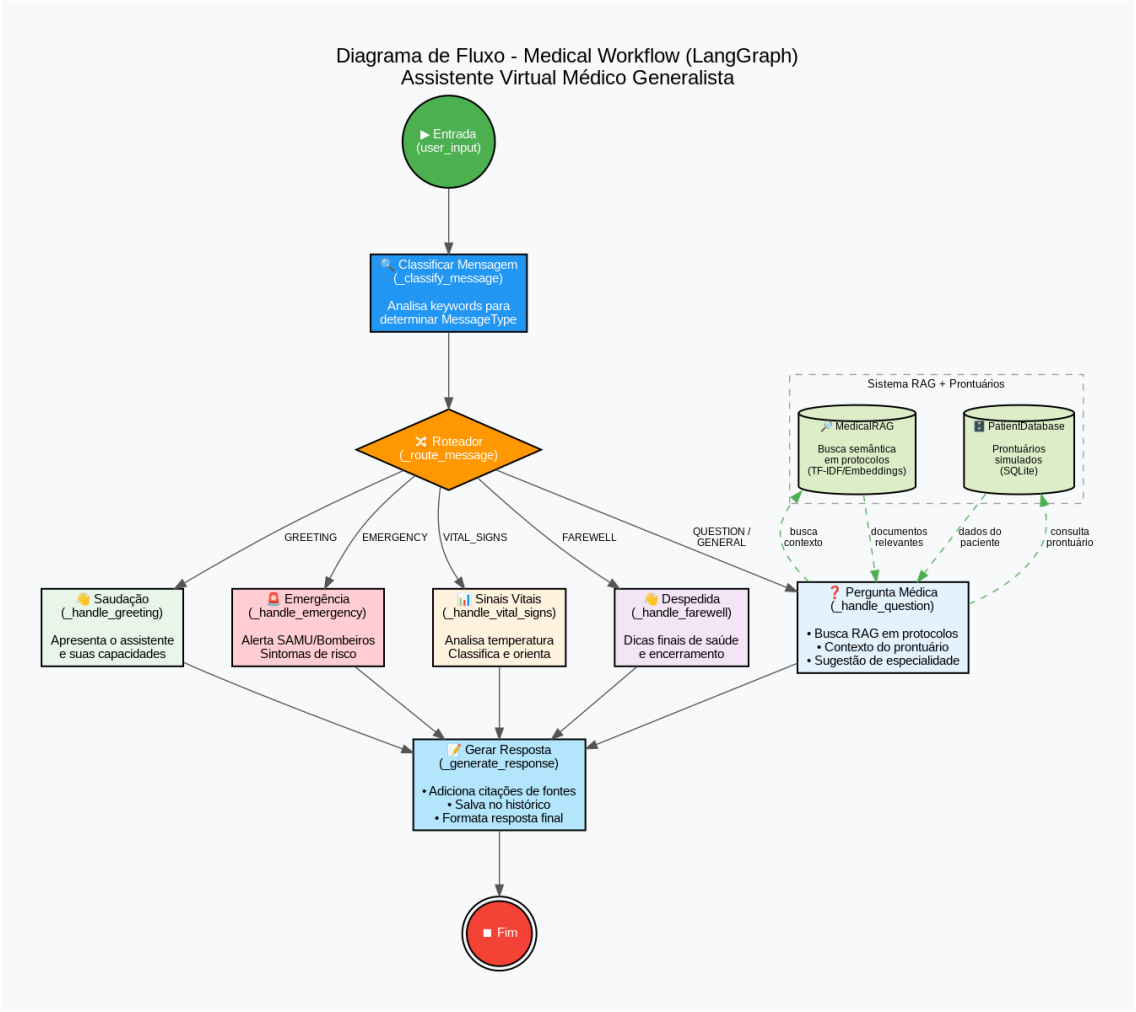

Figura 1 - Arquitetura do Assistente Virtual Médico.
Fonte: elaborada pelos autores (2026).

Após a Figura 1, observa-se que o fluxo do sistema inicia com a classificação da entrada do usuário, seguida pelo roteamento para componentes especializados. Dependendo do tipo de requisição, o sistema pode acionar o mecanismo de recuperação de informação (RAG) e consultar a base de dados de pacientes antes da geração da resposta final pelo modelo de linguagem.

5. DESENVOLVIMENTO

5.1 Fine-Tuning do Modelo

O fine-tuning foi realizado utilizando modelos como BioMistral-7B e TinyLlama, com adaptação por meio da técnica LoRA (Low-Rank Adaptation), combinada com quantização em 4 bits (QLoRA), visando eficiência computacional.

O modelo BioMistral foi selecionado por sua especialização no domínio biomédico, sendo treinado com dados clínicos e científicos, o que proporciona melhor desempenho em tarefas médicas quando comparado a modelos generalistas.

A configuração utilizada no processo de treinamento é apresentada a seguir:

- Técnica: LoRA
- Quantização: 4 bits (QLoRA)
- Rank: 16
- Alpha: 32
- Dropout: 0.05
- Módulos: q_proj, k_proj, v_proj, o_proj

A parametrização do LoRA foi implementada conforme ilustrado no Código 1.

```
LoraConfig(
  r=16,
  lora_alpha=32,
  target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
  lora_dropout=0.05,
  bias="none",
  task_type="CAUSAL_LM",
)
```

Código 1 – Trecho simplificado da configuração do LoRA para fine-tuning do modelo.
Fonte: elaborado pelos autores (2026).

A configuração apresentada demonstra a aplicação da técnica de adaptação de baixa ordem nos módulos de atenção do modelo, permitindo reduzir significativamente o custo computacional sem comprometer a qualidade das respostas geradas.

Pipeline:

1. Carregamento do dataset (JSONL)
2. Tokenização
3. Configuração do modelo
4. Treinamento supervisionado
5. Testes de inferência

5.2 Sistema RAG

O sistema de recuperação foi implementado utilizando a técnica de TF-IDF, permitindo a identificação de documentos relevantes com base na similaridade textual.

As principais configurações incluem:

- n-grams (1,2)
- top_k = 3
- Filtro por relevância

Essa abordagem possibilita a recuperação de protocolos médicos relevantes, enriquecendo o contexto utilizado pelo modelo de linguagem na geração das respostas.

Além disso, o sistema integra informações provenientes do prontuário do paciente, permitindo personalização das respostas com base no histórico clínico. O processo de recuperação de contexto é ilustrado no Código 2.

```
patient_context = None

if patient_id:
    patient_context = patient_db.get_patient_context_for_query(
        patient_id, query
    )

rag_context, sources = rag.get_context_for_query(
    query=query,
    top_k=3,
    patient_context=patient_context,
)
```

Código 2 – Trecho simplificado da recuperação de contexto com RAG e dados do paciente.
Fonte: elaborado pelos autores (2026).

O trecho apresentado demonstra a integração entre a recuperação de dados clínicos do paciente e a busca por protocolos médicos. Essa combinação permite enriquecer o contexto fornecido ao modelo, resultando em respostas mais precisas e contextualizadas.

5.3 Base de Dados

Foi utilizada uma base SQLite contendo prontuários simulados com:

- histórico médico
- medicações
- alergias
- exames

Isso permite que o assistente personalize as respostas com base no histórico clínico do paciente, se um ID de paciente for fornecido.

5.4 Segurança

Foram implementados mecanismos como:

- validação de entrada
- sanitização
- detecção de emergências
- proteção contra injeções

5.5 Explainability

Todas as respostas incluem referências às fontes utilizadas, garantindo transparência e confiabilidade.

---

6. FLUXO DE FUNCIONAMENTO

1. Usuário envia uma pergunta
2. A entrada é validada e sanitizada pela camada de segurança
3. A mensagem é classificada por um nó do LangGraph para determinar a intenção (ex: pergunta médica, saudação, emergência)
4. Sistema roteia para o fluxo adequado
5. Para perguntas médicas, o sistema RAG busca por contexto relevante nos documentos e, se aplicável, no prontuário do paciente
6. Os dados do paciente e o contexto do RAG são incorporados ao prompt
7. O modelo de linguagem gera uma resposta com base no prompt enriquecido
8. As fontes consultadas pelo RAG são anexadas à resposta
9. A resposta final é formatada e retornada ao usuário, e a interação é armazenada no histórico da conversa

---

7. AVALIAÇÃO E RESULTADOS

Métricas utilizadas:

- Perplexidade
- Similaridade de respostas
- Relevance Score (RAG)
- Top-K Precision

Resultados:

- Melhoria na precisão das respostas
- Redução de inconsistências
- Melhor uso de contexto médico

Exemplo de Execução

Entrada:
Paciente com dor no peito e falta de ar

Saída (resumo):
O sistema identifica possível emergência médica, recomenda atendimento imediato e apresenta referências baseadas em protocolos médicos.

---

8. DISCUSSÃO

A combinação de fine-tuning, RAG e orquestração demonstrou ser eficaz para construção de assistentes especializados.

Limitações:

- **Dependência da qualidade dos dados:** A performance do modelo é diretamente influenciada pela qualidade e abrangência dos documentos usados no RAG e no fine-tuning
- **Limitações do TF-IDF:** A busca por TF-IDF pode não capturar a semântica de perguntas curtas ou ambíguas com a mesma eficácia de modelos baseados em embeddings
- **Necessidade de validação humana:** As respostas, embora baseadas em fontes confiáveis, não substituem uma consulta médica e devem ser validadas por um profissional de saúde
- **Requisitos de Hardware:** A inferência com o modelo BioMistral-7B em tempo real requer uma GPU, tornando o custo de implantação um fator a ser considerado

---

9. CONCLUSÃO

O projeto demonstrou a viabilidade de construção de um assistente virtual médico robusto, utilizando técnicas modernas de IA.

Os objetivos foram alcançados, com destaque para:

- pipeline completo de IA
- integração de múltiplos componentes
- foco em segurança e explicabilidade

---

10. TRABALHOS FUTUROS

- Uso de embeddings avançados
- Integração com FHIR/HL7
- Avaliação com métricas como BLEU/ROUGE
- Interface web interativa
- Expansão da Base de Prontuários

---

11. REFERÊNCIAS

- Documentação HuggingFace Transformers
- Documentação PEFT
- Documentação LangChain
- Documentação LangGraph
- Protocolos CONITEC
- TelessaúdeRS